HOCHSCHULE ESSLINGEN

BETRIEBSSYSTEME (BS)

III. BASH PROGRAMMIERUNG (II) – UNIX SHELL UND BASH

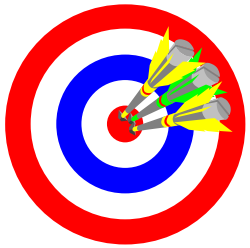
PROF. DR.-ING. RAINER KELLER
PROF. DR. DENNIS GREWE

AGENDA

- Wie entwickle ich Programme für die Bash?
- Was ist ein UNIX Dateisystem?
- Wie finde ich mich im Dateisystem zurecht?

 © Copyright Material. All rights reserved.

ZIELE



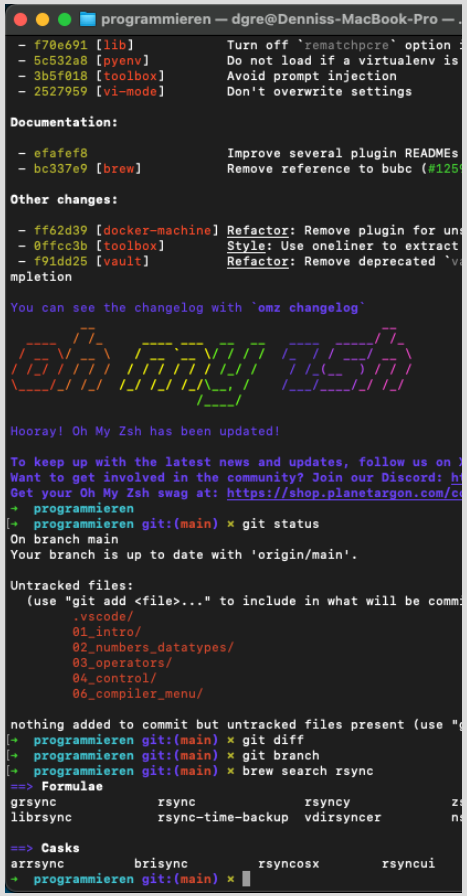
- Entwicklung von kleinen Programmen für die Bash erlernen
- Wichtigste Kommandos in der Bash zur Navigation und Interaktion mit dem Dateisystem.

- HINWEIS: Machen Sie sich Notizen in den jeweiligen Grafiken

© Copyright Material. All rights reserved.

HOCHSCHULE ESSLINGEN

BASH SCRIPTING



Quelle: Dennis Grewe

HINWEIS

- Die nachfolgenden Inhalte sollten Sie direkt in Ihrer Linux-VM oder an einem Linux-Rechner machen

- Code Beispiele finden Sie auf HS Esslingen Gitlab Server -> VPN Verbindung benötigt
  - https://gitlab.hs-esslingen.de/dgrewe/betriebssysteme

© Copyright Material. All rights reserved.

WAS SIND UNIX SHELL SCRIPTE?

HOCHSCHULE ESSLINGEN

- Ein **Shell Script** ist eine Textdatei, welche eine Reihe von Kommandos zu einem ausführbaren Programmablauf zusammenfasst.

- **Beachte**: Alles was normal in der Befehlszeile ausgeführt werden kann, kann zu einem Skript zusammengefasst werden. Ebenso kann alles, was in einem Skript zusammengefasst wurde, auch normal auf der Befehlszeile ausgeführt werden. Das Ergebnis ist beides Mal das selbe.

 © Copyright Material. All rights reserved.

BEISPIELE FÜR SHELL SCRIPTE

HOCHSCHULE ESSLINGEN

- Shell Scripte können einer Vielzahl von Anwendungsfällen dienlich sein:
  - User-Administration (Anlegen, Ändern, Löschen von Benutzern)
  - Starten- und Stoppen von Diensten (Web-, …, Datenbank-Server)
  - Einspielen von Updates einer Anwendung im Rechenzentrum
  - Überprüfen des laufenden Systems und dessen Dienste
  - Erstellen von Backups
  - …

 © Copyright Material. All rights reserved.

FUNKTIONSUMFANG SHELL SCRIPTING (I)

HOCHSCHULE ESSLINGEN

- Die Bash bietet viele vordefinierte Variablen und Funktionen für die Erstellung von Shell-Skripte:
  - Argumente:
    - `$ ./myscript args1 args`
    - `$ echo "Argument one is $1 and two is $2"`
  - Bedingungen:

```
if [[ some_test ]]
then
  # do something
fi
```

```
read -p "What is your name? " name

if [[ -z ${name} ]]
then
  echo "Please enter your name!"
else
  echo "Hi there ${name}"
fi
```

 © Copyright Material. All rights reserved.

FUNKTIONSUMFANG SHELL SCRIPTING (II)

HOCHSCHULE ESSLINGEN

- Switch Case:

```
case $some_variable in
  pattern_1)
    # do something
    ;;
  pattern_2 | pattern_3)
    # something else
    ;;
  *)
    # default commands
    ;;
esac
```

```
read -p ”Enter a car brand: " car
case $car in
  Tesla)
    echo –n “${car}’s factory is in USA.”
    ;;
  BMW | Mercedes | Audi | VW | Porsche)
    echo –n “${car}’s factory is in Germany.”
  *)
    echo -n “${car} is an unknown brand.”
    ;;
esac
```

- Felder/Arrays:
  - `$ my_array=("value 1" "value 2" "value 3" "value 4")`
  - `$ echo ${my_array[1]}`

 © Copyright Material. All rights reserved.

FUNKTIONSUMFANG SHELL SCRIPTING (II)

HOCHSCHULE ESSLINGEN

- For-Schleifen:

```
for var in ${list}
do
  # do something
done
```

```
users="tony amanda john"

for user in ${users}
do
  echo "${user}"
done
```

- While-Schleifen:

```
counter=1
While [[ $counter -le 10]]
do
  # do something
  ((counter++))
done
```

```
read -p "What is your name? " name

while [[ -z ${name} ]]
do
  echo "Name cannot be blank"
  read -p "Enter a name again? " name
done
echo "Hi there ${name}
```

 © Copyright Material. All rights reserved.

FUNKTIONSUMFANG SHELL SCRIPTING (III)

HOCHSCHULE ESSLINGEN

- Bash Funktionen:

```
function function_name() {
  # your commands
}
```

```
function hello() {
  echo "Hello Function!"
}
hello # call the function
```

- Debugging / Testing:
  - Mit dem Befehl set –x kann der Debug-Modus aktiviert werden, um alle ausgeführten Kommandos im Terminal schrittweise auszugeben
  - `$ bash -x ./my_script`

 © Copyright Material. All rights reserved.

REGELN FÜR PROGRAMMIERUNG VON SHELL-SCRIPTEN

HOCHSCHULE ESSLINGEN

- Bei der Erstellung von Shell-Scripten sollten folgende Regeln berücksichtigt werden:
  - **Dokumentieren:** alles hinter einem `# ist ein Kommentar`
  - **Shell festlegen:** in der 1. Zeile – `#!/bin/bash`
  - **Modular programmieren:** KISS und UNIX-Philosophie
  - **Sauber programmieren:** wird ein Parameter erwartet? Sind Dateien verfügbar?...

 © Copyright Material. All rights reserved.

BEISPIEL SHELL SCRIPT

HOCHSCHULE ESSLINGEN

- Skripte erleichtern die Arbeit
  - Skript zur Identifikation aller laufenden Prozesse mit offenen Ports!
  - Minimiert die Anzahl der offenen Netzwerkports -> muss als Superuser / Root ausgeführt werden

```
for PORT in `netstat -nat | grep -E 'LISTEN' |
     cut -f2-4 -d':' |cut -f1 -d' '|sed -e 's/://g'| sort -nu`;
do
  PID=`fuser -n tcp $PORT | cut -c1-6`;
  echo ""; echo "INFORMATION FOR OPEN PORT $PORT:";
  echo "----------------------------";
  ps -f -q $PID
done
```

 © Copyright Material. All rights reserved.

KLAUSURFRAGE WS2024

HOCHSCHULE ESSLINGEN

- Schreiben Sie das Bash Programm `list_user_processes.sh`, welches eine Liste aller Prozesse eines bestimmten Benutzers auf der Konsole ausgibt. Der Aufruf

```
./list_user_processes.sh test
```

des Benutzers “test“ ergibt folgende Ausgabe:

```
test@test-virt-7-2:~$ ./list_user_processes.sh test
PID COMMAND
3035 system
3063 gnome-keyring-d
3066 dbus-daemon
...
```

- Stellen Sie sicher, dass Ihr Skript auf bei der Übergabe eines nicht existierenden Benutzers im System funktioniert.

 © Copyright Material. All rights reserved.

QUIZ ZUR WIEDERHOLUNG: BASH

https://particify.hs-esslingen.de/p/18691236

[Bild von vecstock auf Freepik]

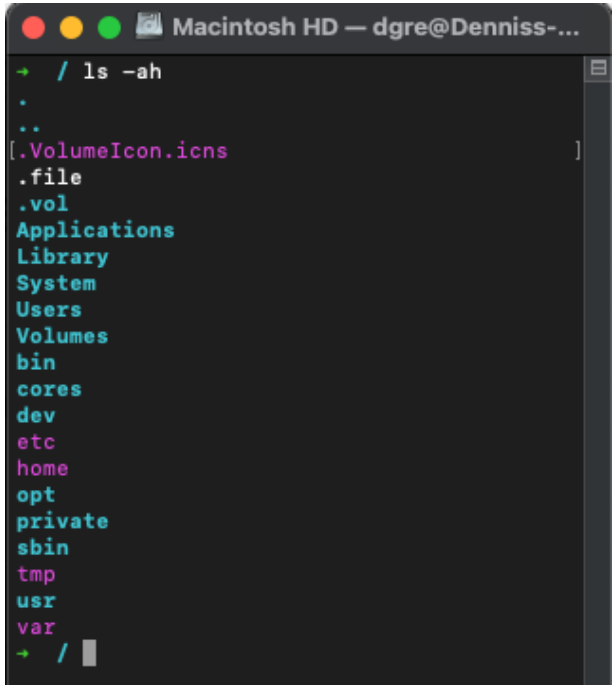
 © Copyright Material. All rights reserved.

UNIX DATEISYSTEMBAUM

Quelle: [Image by freepik]

UNIX DATEISYSTEM (I)

- Das UNIX Dateisystem stellt eine *Virtualisierung des Festplattenspeicher* als hierarchisches Dateisystem (engl. *file system*, oder FS) dar. Dateien sind in einem Verzeichnisbaum eingeordnet



© Copyright Material. All rights reserved.

UNIX DATEISYSTEM (II)

HOCHSCHULE ESSLINGEN

- Die wichtigsten Verzeichnisse im UNIX Dateisystem sind:
  - / → Pfade werden unter Unix durch Slash "`/`" getrennt (Windows mit Backslash "`\`")
  - bin/ → Verzeichnisse mit Namen `bin` enthalten per UNIX-Konvention Binaries, d.h. ausführbare Programme
  - dev/ → Das `/dev/` Verzeichnis enthält Dateien zum Zugriff auf „Devices“, also Geräte, die Hardware, …
  - etc/ → `/etc/` enthält Konfigurationsdateien
  - home/ → In `/home/` liegen alle Benutzerdateien
  - lib/ → Verzeichnisse mit Namen `lib` enthalten per UNIX-Konvention Bibliotheken (engl. *libraries*).
  - tmp/ → Im `/tmp/` Verzeichnis werden temporäre Dateien gespeichert – Achtung: Admin könnte löschen!
  - usr/ → Im `/usr/` Verzeichnis beinhaltet Dateien und installierte Programme (*system-wide, read-only*)

 © Copyright Material. All rights reserved.

WAS SIND DATEISYSTEMRECHTE (FILE PERMISSIONS)?

HOCHSCHULE ESSLINGEN

- UNIX erlaubt das Festlegen von Zugriffsrechten für jede einzelne Datei im Dateisystem um diese zu lesen (engl. *read*), zu verändern (engl. *write*), oder zu sehen (engl. *view*)
- Geben wir den Verzeichnisinhalt „long“ aus:

```
$ ls -al ↵     # List all files in long format
total 200
drwxr-x---+ 30 dgre  staff    960 Aug 16 07:46 .
drwxr-xr-x   5 root  admin    160 Aug 16 07:45 ..
-rw-r--r--   1 dgre  staff   4008 Jul 26 21:46 .bashrc
drwx------@  6 dgre  staff    192 Aug 11 13:43 Desktop
```

© Copyright Material. All rights reserved.

WAS SIND DATEISYSTEMRECHTE (FILE PERMISSIONS)?

HOCHSCHULE ESSLINGEN

```
$ ls -al ↵   # List all files in long format
total 200
drwxr-x---+ 30 dgre  staff    960 Aug 16 07:46 .
drwxr-xr-x   5 root  admin    160 Aug 16 07:45 ..
-rw-r--r--   1 dgre  staff   4008 Jul 26 21:46 .bashrc
drwx------@  6 dgre  staff    192 Aug 11 13:43 Desktop
```

- User
- Group
- Anzahl Verweise
- Größe Bytes
- Datum der letzten Änderung
- Name der Datei oder des Unterverzeichnisses, Namen beginnend mit „.“ werden bei `ls` versteckt, s. `ls -a` um diese zu sehen

1. Typ (`d`=Directory, `-`=File, `l`=softlink)
2. Rechte v. „User“ (r=read, w=write, x=execute)
3. Rechte v. „Group members“ (hier `staff`)
4. Rechte v. „Other“ (auch Welt genannt)

 © Copyright Material. All rights reserved.

ÜBUNG: SETZEN VON DATEISYSTEMRECHTE

HOCHSCHULE ESSLINGEN

- Probieren Sie das Löschen eines Verzeichnis mittels `rm` (remove) auf der Laborumgebung:

```
$ rm -fr /home/rakeller/ ↵          # rm = remove
```

- Warum erhalten Sie eine Fehlermeldung?
- Warum können Sie das Verzeichnis nicht Löschen?

- Mit dem Befehl chmod (Change MODe) können Dateisystemrechte geändert werden:

```
$ chmod 600 ~/.bashrc ↵          # setzt read/write owner
$ ls -al ~/.bashrc ↵             # zum Vergleich Ausgabe im Long Format
```

 © Copyright Material. All rights reserved.

SETZEN VON DATEISYSTEMRECHTE MIT `CHMOD`

HOCHSCHULE ESSLINGEN

- Mit dem Befehl chmod (Change MODe) können Dateisystemrechte geändert werden:

```
$ chmod 600 ~/.bashrc ↵          # setzt read/write owner
```

- Jede Oktalzahl steht für bestimmte Benutzer/Rechte
  - `400`: Der Besitzer (hier: `dgre`) darf die Datei lesen
  - `200`: Der Besitzer (hier: `dgre`) darf die Datei schreiben
  - `100`: Der Besitzer (hier: `dgre`) darf die Datei ausführen
  - `700`: Der Besitzer (hier: `dgre`) darf die Datei lesen (4), schreiben (2) und ausführen (1) – 4+2+1
  - `040`: Die Gruppe (hier: `staff`) darf die Datei lesen
  - `020`: Die Gruppe (hier: `staff`) darf die Datei schreiben
  - `010`: Die Gruppe (hier: `staff`) darf die Datei ausführen
  - `004`: alle anderen Benutzer („World“) dürfen die Datei lesen
  - `002`: alle anderen Benutzer („World“) darf die Datei schreiben
  - `001`: alle anderen Benutzer („World“) darf die Datei ausführen

 © Copyright Material. All rights reserved.

SETZEN VON DATEISYSTEMRECHTE MIT CHMOD

HOCHSCHULE ESSLINGEN

- Mit dem Befehl chmod (Change MODe) können Dateisystemrechte geändert werden:

```
$ chmod 600 ~/.bashrc ↵          # setzt read/write owner
```

- Für die obige Befehlszeile heißt das:
  - 1. Zahl (6) gibt an, dass Owner Lese-& Schreib-Rechte erhält,
  - 2. Zahl (0) gibt an, dass die Gruppe „staff“ **und**
  - 3. Zahl (0) alle anderen (“Welt“) keine Read-/Write-/Execute-Rechte haben!

- Was würde der Wert 005 bedeuten?
- Um das „Home“-Verzeichnis nicht zugreifbar für „Welt“ zu machen:

```
$ chmod 700 ~ ↵          # zurücksetzen von HOME auf Zugriff nur für Owner
```

 © Copyright Material. All rights reserved.

KLAUSURFRAGE WS2024 (2,5 PUNKTE)

HOCHSCHULE ESSLINGEN

- Folgende Rechte sind in einem Datei-Listing vermerkt: `-rw-r-----` Was bedeuten diese Rechte? Erklären Sie und gehen Sie dabei sowohl auf die besetzten als auch auf die nicht gesetzten Bits (-) ein. Geben Sie die Rechte auch in der numerischen Notation (wie z.B. 755) an.

 © Copyright Material. All rights reserved.

SETZEN VON DATEISYSTEMRECHTE MIT CHMOD

HOCHSCHULE ESSLINGEN

- Die Rechte lassen sich auch „lesbar“ formulieren:Dateisystemrechte geändert werden:
  - Für Besitzer (user) schreibt man „u“,
  - Für Gruppe (group) schreibt man „g“,
  - Für Andere (other) schreibt man „o“
- Mit „+“ fügt man weitere Rechte hinzu, mit „-“ werden diese abgezogen, mit „=“ exakt gesetzt.
- Mit Lesen „r“, Schreiben „w“ und Ausführen „x“ ergibt sich:

```
$ chmod gu=rw ~/.bashrc ↵        # exakt rw&wr user & group
$ chmod o+x ~/.bashrc ↵          # 'other' darf ausführen
$ chmod g-w ~/.bashrc ↵          # 'group' darf nicht schreiben
```

 © Copyright Material. All rights reserved.

DATEISYSTEMRECHTE FÜR VERZEICHNISSE

HOCHSCHULE ESSLINGEN

- Für Dateien ist es nun klar, was die Rechte bedeuten. Wie sieht es aber mit Verzeichnissen(engl. *directories*) aus?
  - Lesen: der Verzeichnisinhalt darf angezeigt werden (`ls` Befehl)
  - Schreiben: Es dürfen Dateien/Verzeichnisse erzeugt werden
  - Ausführen: in das Verzeichnis darf gewechselt werden (`cd` Befehl)
- **Achtung**: Es macht also keinen Sinn für *Owner* Lese- oder Schreibrechte für ein Verzeichnis zu verbieten! Für die Gruppe ‘group‘ oder Andere ‘other‘ (World) allerding schon.

 © Copyright Material. All rights reserved.

ÜBUNG: DATEISYSTEMRECHTE FÜR VERZEICHNISSE

HOCHSCHULE ESSLINGEN

- **Aufgabe**: Setzen sie Ihr “Home“-Verzeichnis auf nicht-schreib- und ausführbar für die Benutzergruppe ‘other‘ (World). Erlauben Sie aber das Lesen! Probieren Sie anschließend gegenseitig, ob sie eine (lesbare) Datei in Ihr Verzeichnis kopieren können?

- **Hinweis**: Für das Kopieren einer Datei in der Shell können Sie den Befehl `cp` (copy) nutzen.
  - https://man7.org/linux/man-pages/man1/cp.1.html

 © Copyright Material. All rights reserved.

UNIX DATEIINFORMATIONEN

HOCHSCHULE ESSLINGEN

- UNIX speichert exakt die Informationen wann Dateien/Ordern/…
  - angelegt wurden (created)
  - diese das letzte mal gelesen wurden (accessed)
  - diese das letzte mal verändert wurden (modified)
- Es muss daher gelten:
  - $t_{Created} <= t_{Modified} <= t_{Accessed}$

- Eine weitere Möglichkeit für das Setzen von Dateisystemrechten in UNIX sind sogenannte Access Control Lists (ACL). Diese behandeln wir aber in dieser Veranstaltung nicht!

 © Copyright Material. All rights reserved.

HOCHSCHULE ESSLINGEN

WEITERE UNIX BEFEHLE

LERNEN SIE DAS CHEAT-SHEET

Quelle: [Image by rawpixel.com on Freepik]

UNIX VERZEICHNISSE ANLEGEN MIT `MKDIR`

HOCHSCHULE ESSLINGEN

- Mit dem Befehl `mkdir` (**m**ak**e** **dir**ectory) können neue Verzeichnisse angelegt werden:
  - `$ mkdir mein_verzeichnis ↵  # make directory`
- Existiert das Verzeichnis bereits, so wird eine Fehlermeldung angezeigt.

- Mit dem Parameter `-p` (für parent) können Sie einen ganzen Verzeichnisbaum anlegen:
  - `$ mkdir -p mein_dir/src ↵  # make directory`
- Es wird kein Fehler angezeigt, wenn bereits eines der Verzeichnisse existiert.

 © Copyright Material. All rights reserved.

WAS SIND “HARDLINKS“?

HOCHSCHULE ESSLINGEN

- Mit sogenannten *Links* können Sie *Verweise* im UNIX Dateisystem anlegen.
- **Hardlinks** verweisen in UNIX direkt auf andere Dateien innerhalb des Dateisystems (also einer gemounteten Partition). Beim Zugriff auf den Hardlink verweist das Betriebssystem direkt auf die eigentliche Datei.

- Hardlinks können mit dem Befehl `ln` angelegt werden:
  - `$ ln echte_datei my_link ↵  # legt hard link ‘my_link‘ an`
- Der Link `my_link` ist nicht von der eigentlichen Datei unterscheidbar, d.h. beim Aufrufen wird direkt die echte Datei geöffnet.
- **Nachteil**: Kann nicht über Dateisystemgrenzen angelegt werden (z.B.: falls `/home` NFS, und `/etc` ext2 System)

 © Copyright Material. All rights reserved.

WAS SIND “SOFTLINKS“?

HOCHSCHULE ESSLINGEN

- **Softlinks** verweisen in UNIX auf andere Dateien innerhalb des Dateisystems. Beim Zugriff auf den Softlink lenkt das Betriebssystem auf die eigentliche Datei um und gleicht den Nachteil der Hardlinks aus.
- Softlinks können mit dem Befehl `ln` und dem Zusatz `-s` angelegt werden:

```
$ ln -s tmp.txt my_link ↵      # legt softlink 'my_link' an
$ ls -l ↵
-rw-r--r-- rakeller staff 18 Apr 16:14 tmp.txt
lrwxrwxrwx rakeller staff 18 Apr 16:14 my_link->tmp.txt
```

- Softlinks eignen sich gut, wenn man eine Datei in verschiedene Verzeichnisse einordnen kann: `~/Dokumente` und `~/Arbeit`.

 © Copyright Material. All rights reserved.

WAS SIND “DEVICE FILES“?

HOCHSCHULE ESSLINGEN

- **Device Files** sind Geräte-Dateien, welche die Schnittstelle von Systemanwendungen zur Konfiguration von Geräten darstellen. Das System legt diese Dateien meist automatisch im Verzeichnis /dev an:
- Softlinks können mit dem Befehl `ln` und dem Zusatz `-s` angelegt werden:

```
$ ls -al /dev ↵      # Ausgabe des /dev Verzeichnis
brw-r-----  root  operator 14, 0 22 Apr 16:00 disk0
```

b=Block Device

Gerätenummer X,Y:
X=Major Number
Y=Minor Number
Das ist OS-Spezifisch.
Hier 14 = 1. Platte
Hier 0 = Gesamte Platte
(1 = 1. Partition, usw.)

- Das Systemprogramm mknod erzeugt diesen Dateityp:

```
$ mknod /dev/disk0 -b 14 0 ↵     # Erzeugt Gerätedatei
```

 © Copyright Material. All rights reserved.

DATEISSYSTEMVERWALTUNG – MOUNT, MKFS UND FSCK

HOCHSCHULE ESSLINGEN

- Mit dem Befehl mount lassen sich Verzeichnisbäume in ein Dateisystem einhängen.
  - `$ mount -t fs_type device mount_point ↵`
- Zum Beispiel um eine CD (FS iso9660) des Gerätes `/dev/cdrom` einzuhängen:
  - `$ mount -t iso9660 /dev/cdrom /mnt/my_cd ↵`

- Um ein Dateisystem zu erzeugen gibt es den `mkfs.XXX` Befehl:
  - `$ mkfs.ext3 device ↵  # Erzeugt ein FS vom Typ ext3 auf dem Gerät 'device'`

- Um ein Dateisystem auf Fehler zu überprüfen gibt es den `fsck` Befehl:
  - `$ fsck device ↵  # file system check`

 © Copyright Material. All rights reserved.

BEISPIEL: FILE MANAGEMENT SCRIPT

HOCHSCHULE ESSLINGEN

- **Aufgabe**: Schreiben Sie ein Bash-Programm, welches einem Benutzer das Erstellen, Löschen von Dateien, sowie die Darstellung der Inhalte von Dateien erlaubt.
  - Der Benutzer soll Befehle über ein einfaches Shell-Menü eingeben.
  - Nach dem Erstellen einer Datei soll der Benutzer ebenfalls in der Lage sein Inhalte in der neu erzeugten Datei zu speichern.

- *Hinweis*: Sie benötigen Funktionen, Bedingungen und Schleifenkonstrukte für die Umsetzung des Programmablaufs, sowie Befehle im Umgang mit dem Dateisystembaum.

 © Copyright Material. All rights reserved.

ZUSAMMENFASSUNG

HOCHSCHULE ESSLINGEN

- Kennenlernen der Bash und des Funktionsumfangs.
- Kennenlernen der Bash Scripting Funktionalität.
- Kennenlernen und Umgang mit dem UNIX Dateisystem und Dateisystemrechten.
- Kennenlernen von Verweisen (Hard- und Softlinks) und Einbinden von Geräten mittels mount Befehl.

 © Copyright Material. All rights reserved.